The Race Condition Problem — Complete Deep Dive

What is Actually Happening Here

Let me show you exactly what happens when you run this code.

Both `writeFile` and `appendFile` are **asynchronous.** This means the moment Node.js sees them, it does NOT wait. It hands them off to libuv's thread pool and **immediately moves to the next line.**

So the execution looks like this:

Line 1: fs.writeFile(...)
→ handed to thread pool background thread
→ Node.js does NOT wait
→ immediately moves to next line

Line 2: fs.appendFile(...)
→ handed to thread pool background thread
→ Node.js does NOT wait
→ immediately moves to next line

Line 3: console.log('File Operation done')
→ prints immediately

So `console.log('File Operation done')` prints **before either file operation completes.** That is the first surprise.

The Real Problem — Two Threads Fighting Over One File

Now here is the dangerous part.

After Node.js fires off both operations, you have this situation:

```
Background Thread 1:          Background Thread 2:
═══════════════════════════════════════════════════
Doing writeFile...            Doing appendFile...
writing 'Hello NodeJS!\n'     trying to append
                              'Hello JavaScript!\n'
═══════════════════════════════════════════════════
```

Two background threads are working on the **same file simultaneously.**

Neither thread knows the other exists. Neither waits for the other. They are both just doing their job.

The Three Possible Outcomes

Because these two operations run simultaneously with no guaranteed order, you can get three different outcomes every time you run the code.

Outcome 1 — writeFile finishes first (what you want)

```
Thread 1 writes:   Hello NodeJS!\n         ← file contains this
Thread 2 appends:  Hello JavaScript!\n     ← appended after

Final file:
Hello NodeJS!
Hello JavaScript!
```

This is the correct result. But it only happens if Thread 1 wins the race.

Outcome 2 — appendFile finishes first (wrong order)

```
Thread 2 appends first to an empty/non-existent file
Thread 1 then writes and OVERWRITES everything

Final file:
Hello NodeJS!
```

The append happened but then writeFile wiped it. `Hello JavaScript!` is gone forever.

Outcome 3 — Both access file at exact same moment (corruption)

Both threads read file state simultaneously
Both write simultaneously
Data gets interleaved or corrupted

Final file:
HHello NelloodeJS!
 JaNodevaJS!script!

Garbage. The file is completely corrupted.

This is Called a Race Condition

The educator was describing exactly this. A **race condition** happens when:

- Two or more operations run simultaneously
- They access the same shared resource (the file)
- The final result depends on which one finishes first
- You have no control over which one wins the race

The word "race" is perfect here. Both operations are literally racing to access the same file. Whoever gets there first determines what your file looks like. And you cannot predict or control who wins.

This is one of the most dangerous bugs in programming because:

- It does not happen every time
- Sometimes your code works perfectly
- Sometimes it produces wrong results
- Sometimes it corrupts data
- It is extremely hard to reproduce consistently
- It is extremely hard to debug

Why This Happens — The Root Cause

The root cause is a fundamental misunderstanding of async code.

When you write:

```
fs.writeFile(pathname, 'Hello NodeJS!\n', callback)
fs.appendFile(pathname, 'Hello JavaScript!\n', callback)
```

You are thinking:

"writeFile will run, finish, then appendFile will run"

But async does NOT work like that. Async means:

"writeFile will START, then appendFile will START immediately,
both run simultaneously, finish in unpredictable order"

Starting is not finishing. Both operations start almost simultaneously. Finishing order is determined by the OS, the disk, the thread scheduler — things completely outside your control.

The Fix — Nest the appendFile Inside the writeFile Callback

The callback of `writeFile` only runs **after writeFile has completely finished.** So if you put `appendFile` inside that callback, you guarantee the order.

```
fs.writeFile(pathname, 'Hello NodeJS!\n', (error) => {
    if (error) {
        console.log("Something went wrong while writing")
        return
    }

    console.log("File written successfully")

    // appendFile only STARTS after writeFile COMPLETELY FINISHES
    fs.appendFile(pathname, 'Hello JavaScript!\n', (error) => {
        if (error) {
            console.log("Something went wrong while appending")
            return
        }

        console.log("File appended successfully")
```

```
    })
})

console.log('File Operation done')
```

Now the flow is:

1. writeFile STARTS
   → handed to background thread

2. console.log('File Operation done') prints immediately

3. writeFile FINISHES
   → callback runs
   → "File written successfully" prints
   → appendFile STARTS (only now, after write is done)

4. appendFile FINISHES
   → "File appended successfully" prints

Final file guaranteed to be:
Hello NodeJS!
Hello JavaScript!

No race condition. No unpredictable order. The nesting enforces the sequence.

Visualising the Difference

Without Nesting — Race Condition

```
Time 0ms:   writeFile starts  ──────────────────┐
Time 0ms:   appendFile starts ────────────────┐  |
Time 0ms:   console.log prints               |  |
                                             |  |
Time ???:   appendFile finishes ──────────────┘  |  ← could be first
Time ???:   writeFile finishes  ─────────────────┘  ← could be first

Result: UNPREDICTABLE
```

With Nesting — Guaranteed Order

```
Time 0ms:    writeFile starts
Time 0ms:    console.log prints

Time 50ms:   writeFile finishes
             ↓
             appendFile starts   ← only starts AFTER write done

Time 100ms:  appendFile finishes
```

Result: GUARANTEED CORRECT ORDER

The Output You Were Getting

The educator mentioned "pt!" appearing in the output. This is exactly the corruption from Outcome 3 — the two threads writing to the file simultaneously, their data getting interleaved, producing garbled content.

That corrupted output is direct proof that both operations were running simultaneously and conflicting with each other.

The Simple Mental Model

Think of the file as a whiteboard in a room.

Without nesting: Two people enter the room at the same time. Both start writing on the whiteboard simultaneously. They bump into each other, write over each other's work, and the result is a mess.

With nesting: Person 1 enters the room, writes on the whiteboard, and leaves. Only after Person 1 leaves does Person 2 enter and add their writing below. Clean, ordered, correct.

The callback nesting is how you make Person 2 wait outside until Person 1 is completely done.

Summary

	Without Nesting	With Nesting
Both operations start	Simultaneously	Sequentially
Order guaranteed	No	Yes
Race condition	Yes	No
File corruption possible	Yes	No
Result	Unpredictable	Always correct

The educator's point was this: **async does not mean sequential.** It means concurrent. If you need operations to happen in a specific order, you must explicitly enforce that order — either through nesting callbacks, chaining promises, or using await. Simply writing one line after another in your code does not guarantee one operation finishes before the next one starts.